

Factors

SISBID: Data Wrangling in R

Factors

A **factor** is a special **character** vector where the elements have pre-defined groups or 'levels'. You can think of these as qualitative or categorical variables:

```
x <- c("yellow", "red", "red", "blue", "yellow", "blue")  
class(x)
```

```
## [1] "character"
```

```
x_fact <- factor(x) # factor() is a function  
class(x_fact)
```

```
## [1] "factor"
```

Factors

Factors have **levels** (character types do not).

```
x  
## [1] "yellow" "red"    "red"    "blue"   "yellow" "blue"
```

```
x_fact  
## [1] yellow red    red    blue   yellow blue  
## Levels: blue red yellow
```

Note that levels are, by default, in **alphanumerical** order.

Factors - get the levels

Extract the levels of a factor vector using `levels()`:

```
levels(x_fact)
```

```
## [1] "blue"  "red"   "yellow"
```

Load the UFO data

```
ufo <-  
  read_csv("https://raw.githubusercontent.com/SISBID/Module1/gh-pages/data/ufo/ufo_data_complete.csv")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details, e.g.:
```

```
##   dat <- vroom(...)
```

```
##   problems(dat)
```

```
## Rows: 88875 Columns: 11
```

```
## — Column specification —————
```

```
## Delimiter: ","
```

```
## chr (10): datetime, city, state, country, shape, duration (hours/min), comme...
```

```
## dbl (1): duration (seconds)
```

```
##
```

```
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Clean the names again

Using janitor.

```
ufo <- clean_names(ufo)
glimpse(ufo)
```

```
## Rows: 88,875
## Columns: 11
## $ datetime      <chr> "10/10/1949 20:30", "10/10/1949 21:00", "10/10/1955...
## $ city          <chr> "san marcos", "lackland afb", "chester (uk/england)...
## $ state         <chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "...
## $ country       <chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us",...
## $ shape         <chr> "cylinder", "light", "circle", "circle", "light", "...
## $ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...
## $ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...
## $ comments      <chr> "This event took place in early fall around 1949-50...
## $ date_posted   <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...
## $ latitude      <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21...
## $ longitude     <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458..."
```

Preparing the data: separate into date and time

```
ufo <- ufo %>%separate(datetime,into = c("date", "time"), sep = " ")
```

```
glimpse(ufo)
```

```
## Rows: 88,875
## Columns: 12
## $ date      <chr> "10/10/1949", "10/10/1949", "10/10/1955", "10/10/19...
## $ time      <chr> "20:30", "21:00", "17:00", "21:00", "20:00", "19:00...
## $ city      <chr> "san marcos", "lackland afb", "chester (uk/england)...
## $ state     <chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "...
## $ country   <chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us",...
## $ shape     <chr> "cylinder", "light", "circle", "circle", "light", "...
## $ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...
## $ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...
## $ comments  <chr> "This event took place in early fall around 1949-50...
## $ date_posted <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...
## $ latitude  <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21...
## $ longitude <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458..."
```

Preparing the data: separate into hour and min

```
ufo <- ufo %>% separate(time, into= c("hour", "min"))
```

```
glimpse(ufo)
```

```
## Rows: 88,875
## Columns: 13
## $ date      <chr> "10/10/1949", "10/10/1949", "10/10/1955", "10/10/19...
## $ hour      <chr> "20", "21", "17", "21", "20", "19", "21", "23", "20...
## $ min       <chr> "30", "00", "00", "00", "00", "00", "00", "45", "00...
## $ city      <chr> "san marcos", "lackland afb", "chester (uk/england)...
## $ state     <chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "...
## $ country   <chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us",...
## $ shape     <chr> "cylinder", "light", "circle", "circle", "light", "...
## $ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...
## $ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...
## $ comments  <chr> "This event took place in early fall around 1949-50...
## $ date_posted <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...
## $ latitude  <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21...
## $ longitude <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458..."
```

Preparing the data: convert to numeric

```
ufo <-ufo %>% mutate(hour = as.numeric(hour), min = as.numeric(min))
```

```
glimpse(ufo)
```

```
## Rows: 88,875
## Columns: 13
## $ date      <chr> "10/10/1949", "10/10/1949", "10/10/1955", "10/10/19...
## $ hour      <dbl> 20, 21, 17, 21, 20, 19, 21, 23, 20, 21, 13, 19, 16,...
## $ min       <dbl> 30, 0, 0, 0, 0, 0, 0, 45, 0, 0, 0, 0, 0, 0, 0, 3...
## $ city      <chr> "san marcos", "lackland afb", "chester (uk/england)...
## $ state     <chr> "tx", "tx", NA, "tx", "hi", "tn", NA, "ct", "al", "...
## $ country   <chr> "us", NA, "gb", "us", "us", "us", "gb", "us", "us",...
## $ shape     <chr> "cylinder", "light", "circle", "circle", "light", "...
## $ duration_seconds <dbl> 2700, 7200, 20, 20, 900, 300, 180, 1200, 180, 120, ...
## $ duration_hours_min <chr> "45 minutes", "1-2 hrs", "20 seconds", "1/2 hour", ...
## $ comments  <chr> "This event took place in early fall around 1949-50...
## $ date_posted <chr> "4/27/2004", "12/16/2005", "1/21/2008", "1/17/2004"...
## $ latitude  <chr> "29.8830556", "29.38421", "53.2", "28.9783333", "21...
## $ longitude <chr> "-97.9411111", "-98.581082", "-2.916667", "-96.6458..."
```

Preparing the data: create timespan variable

```
ufo <- ufo %>% mutate(timespan =  
  case_when(hour %in%c(18,19,20,21)~ "Evening",  
            hour >21 ~ "Night",  
            hour >=0 & hour <12 ~ "Morning",  
            hour >=12 & hour <18 ~ "Afternoon"))  
  
ufo %>% count(timespan)
```

```
## # A tibble: 4 × 2  
##   timespan      n  
##   <chr>    <int>  
## 1 Afternoon 10828  
## 2 Evening   32684  
## 3 Morning   23517  
## 4 Night     21846
```

Check the output

You may have noticed that character data is arranged alphabetically.

```
ufo %>% count(timespan)
```

```
## # A tibble: 4 × 2
##   timespan      n
##   <chr>      <int>
## 1 Afternoon 10828
## 2 Evening   32684
## 3 Morning   23517
## 4 Night     21846
```

Check the output

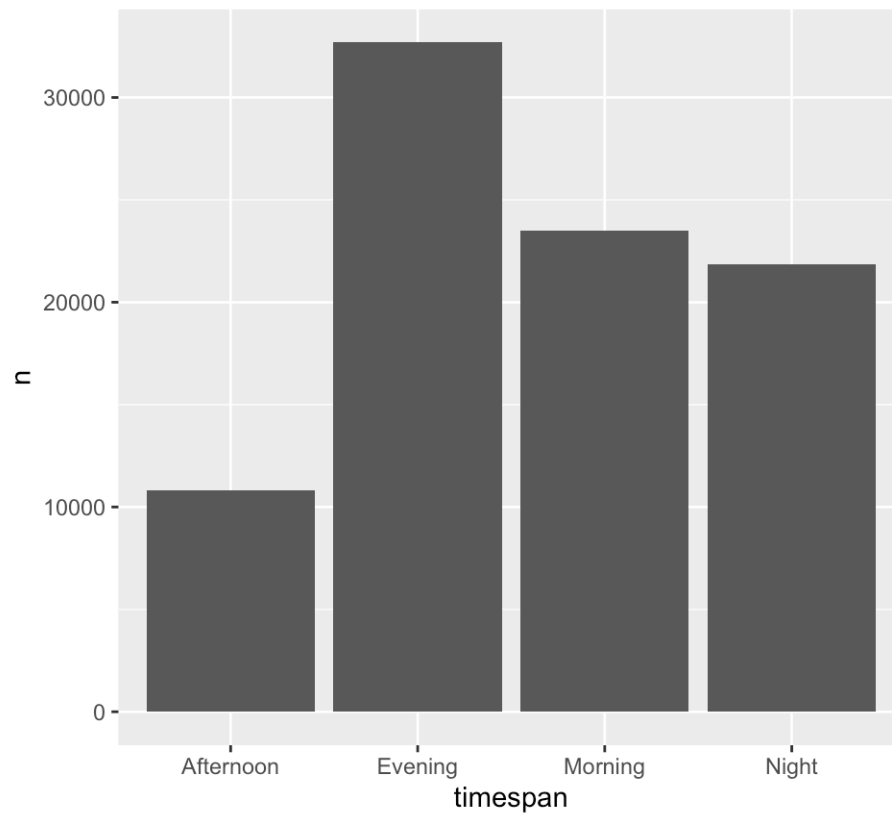
Character data is arranged alphabetically.

```
ufo %>% arrange(timespan) %>% glimpse()
```

```
## Rows: 88,875
## Columns: 14
## $ date          <chr> "10/10/1955", "10/10/1968", "10/10/1970", "10/10/19...
## $ hour          <dbl> 17, 13, 16, 17, 17, 12, 12, 17, 15, 17, 16, 17, 13,...
## $ min           <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 5, 0, 0, 15, 30, 30, 40,...
## $ city          <chr> "chester (uk/england)", "hawthorne", "bellmore", "w...
## $ state         <chr> NA, "ca", "ny", "az", "sc", "tx", "mi", "fl", "tx",...
## $ country       <chr> "gb", "us", "us", NA, "us", "us", "us", "us", NA, N...
## $ shape         <chr> "circle", "circle", "disk", "light", "light", "othe...
## $ duration_seconds <dbl> 20, 300, 1800, 120, 360, 30, 120, 3600, 3600, 0, 14...
## $ duration_hours_min <chr> "20 seconds", "5 min.", "30 min.", "2 min", "5-6 mi...
## $ comments      <chr> "Green/Orange circular disc over Chester&#44 Englan...
## $ date_posted   <chr> "1/21/2008", "10/31/2003", "5/11/2000", "2/18/2001"...
## $ latitude      <chr> "53.2", "33.9163889", "40.6686111", "0", "32.854444...
## $ longitude     <chr> "-2.916667", "-118.3516667", "-73.5275000", "0", "-...
## $ timespan      <chr> "Afternoon", "Afternoon", "Afternoon", "Afternoon",...
```

Let's plot the data

```
count(ufo, timespan) %>% ggplot(aes(x = timespan, y = n)) + geom_col()
```



Let's try that again using factors

Currently `timespan` is class `character` but let's change that to class `factor` which allows us to specify the levels or order of the values.

```
ufo <- ufo %>% mutate(timespan = factor(timespan, levels =  
  c("Morning", "Afternoon", "Evening", "Night")))
```

Now let's take a look at the data again

Factor data is arranged by level.

```
ufo %>% count(timespan)
```

```
## # A tibble: 4 × 2
##   timespan      n
##   <fct>      <int>
## 1 Morning    23517
## 2 Afternoon  10828
## 3 Evening    32684
## 4 Night      21846
```

Nice this is better!

Now let's take a look at the data again

Factor data is arranged by level.

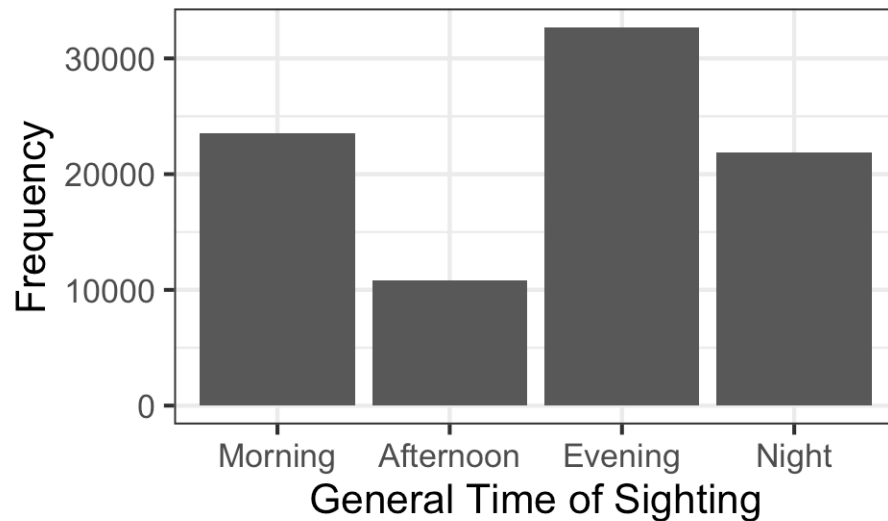
```
ufo %>% arrange(timespan) %>% glimpse()
```

```
## Rows: 88,875
## Columns: 14
## $ date      <chr> "10/10/1978", "10/10/1979", "10/10/1982", "10/10/19...
## $ hour      <dbl> 2, 0, 7, 5, 0, 3, 3, 2, 2, 3, 0, 3, 4, 6, 11, 3, 3,...
## $ min       <dbl> 0, 0, 0, 0, 0, 0, 20, 0, 30, 30, 1, 0, 0, 0, 0, 0, ...
## $ city      <chr> "elmont", "poughkeepsie", "gisborne (new zealand)",...
## $ state     <chr> "ny", "ny", NA, "tx", "ca", NA, "mo", NA, "ca", "az...
## $ country   <chr> "us", "us", NA, "us", "us", NA, "us", NA, "us", "us...
## $ shape     <chr> "rectangle", "chevron", "disk", "circle", "disk", "...
## $ duration_seconds <dbl> 300, 900, 120, 60, 300, 1200, 3, 15, 300, 15, 3600,...
## $ duration_hours_min <chr> "5min", "15 minutes", "2min", "1 minute", "approx 5...
## $ comments  <chr> "A memory I will never forget that happened meny ye...
## $ date_posted <chr> "2/1/2007", "4/16/2005", "1/11/2002", "4/18/2012", ...
## $ latitude  <chr> "40.7008333", "41.7002778", "-38.662334", "29.76305...
## $ longitude <chr> "-73.7133333", "-73.9213889", "178.017649", "-95.36...
## $ timespan  <fct> Morning, Morning, Morning, Morning, Morning, Mornin...
```

Let's plot that again

The factor data is automatically plotted in the order we would like.

```
count(ufo, timespan) %>% ggplot(  
  aes(x = timespan, y = n)) +  
  geom_col() +  
  xlab("General Time of Sighting") + ylab("Frequency")+  
  theme_bw(base_size = 16)
```



forcats package

A package called `forcats` is really helpful for working with factors.



forcats for ordering

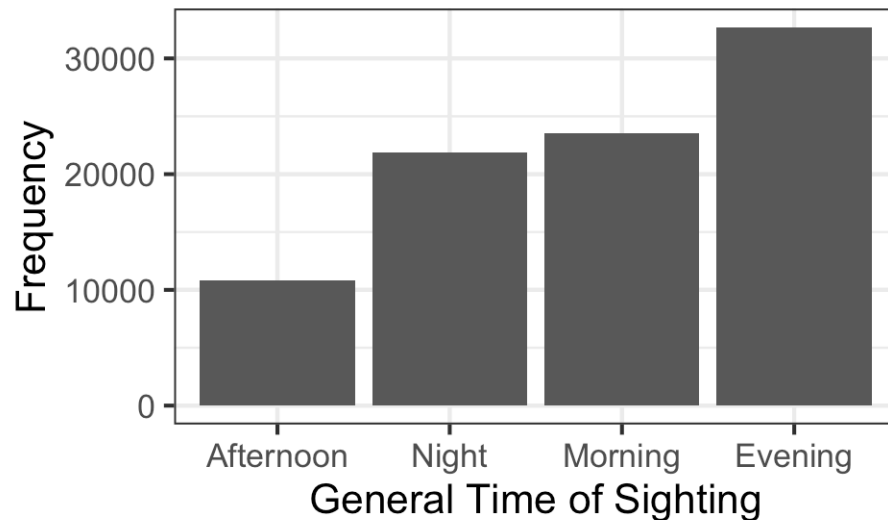
We can order a factor by another variable by using the `fct_reorder()` function of the `forcats` package.

```
fct_reorder({column getting changed}, {guiding column}, {summarizing function})
```

forcats for ordering

Let's reorder by another variable using `fct_reorder`.

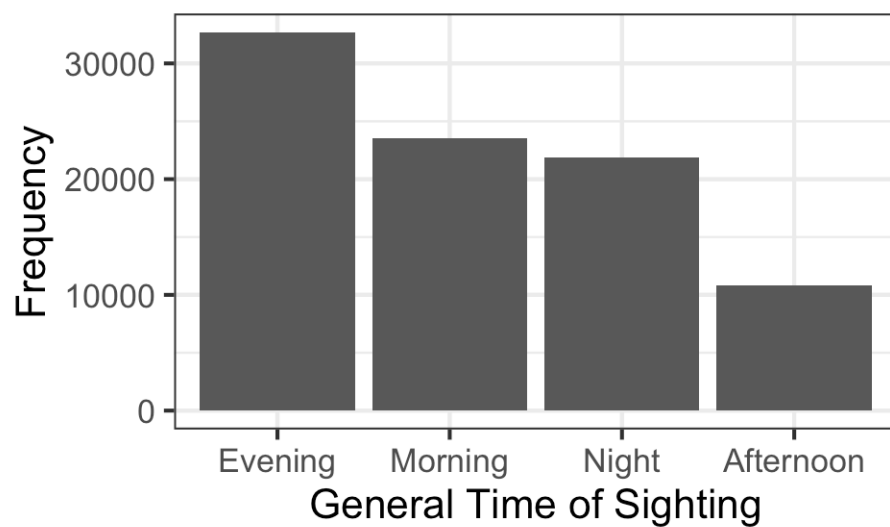
```
count(ufo, timespan) %>% ggplot(  
  aes(x = fct_reorder(timespan, n), y = n)) +  
  geom_col() +  
  xlab("General Time of Sighting") + ylab("Frequency")+  
  theme_bw(base_size = 16)
```



This would be useful for identifying easily which timespan to focus on.

forcats for ordering.. with `.desc` = argument

```
count(ufo, timespan) %>% ggplot(  
  aes(x = fct_reorder(timespan, n, .desc = TRUE), y = n)) +  
  geom_col() +  
  xlab("General Time of Sighting") + ylab("Frequency")+  
  theme_bw(base_size = 16)
```



GUT CHECK: Why use factors?

- A. Meaningful ordering of text data
- B. Automatic ordering of numeric data
- C. More precise values

Checking Proportions with `fct_count()`

The `fct_count()` function of the `forcats` package is helpful for checking that the proportions of each level for a factor are similar. Need the `prop = TRUE` argument otherwise just counts are reported.

```
ufo %>%  
  pull(timespan) %>%  
  fct_count(prop = TRUE)
```

```
## # A tibble: 4 × 3  
##   f          n      p  
##   <fct>    <int> <dbl>  
## 1 Morning  23517 0.265  
## 2 Afternoon 10828 0.122  
## 3 Evening  32684 0.368  
## 4 Night    21846 0.246
```

Summary

- the factor class allows us to have a different order from alphanumeric for categorical data
- we can change data to be a factor variable using `mutate` and a factor creating function like `factor()` (alphabetical order by default)
- with `factor()` we can specify the levels with the `levels` argument if we want a specific order
- the `fct_reorder({variable_to_reorder}, {variable_to_order_by}, {summary function})` helps us reorder a variable by the values of another variable
- arranging, tabulating, and plotting the data will reflect the new order

Lab



[Class Website](#)
[Lab](#)